

Bloc 2 MVC PHP TD4

Contexte	2
Exercice 1	2
Exercice 2	4
Exercice 3	4
Exercice 4	5
Exercice 5	6
Exercice 6	7
Exercice 7	9
Conclusion	11

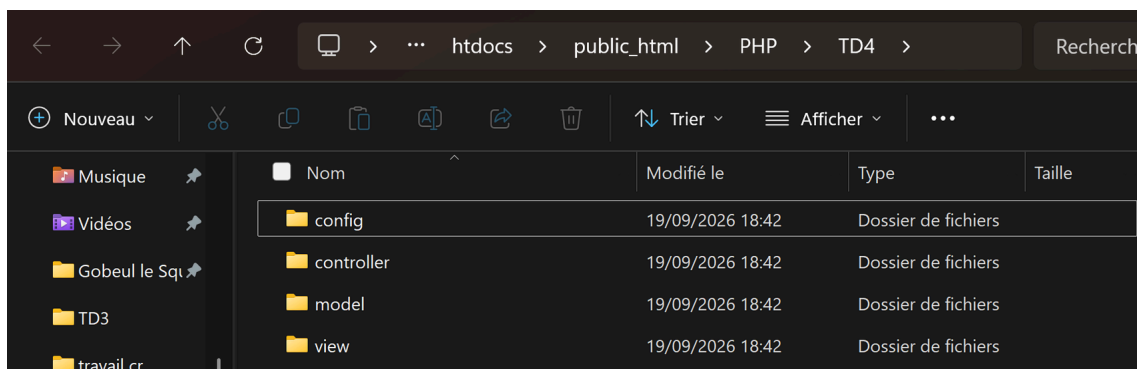
Contexte

Ce **TD** est la suite du **TD3**, il nous fait découvrir l'architecture **MVC** (Modèle, Vue, Contrôleur) pour mieux organiser le code de notre site de covoiturage.

(Les fichiers projet seront disponibles dans le portfolio sur GitHub).

Exercice 1

Création du dossier **"/TD4"** avec les répertoires **"config"**, **"controller"**, **"model"**, **"view"** et **"view/voiture"** :



On renomme **"Voiture.php"** en **"ModelVoiture.php"** ainsi que la classe en **"ModelVoiture"** :

```
<?php
require_once 'Model.php';

class ModelVoiture {

    private $marque;
    private $couleur;
    private $immatriculation;
```

On met la fonction **"afficher()"** en commentaire pour la désactiver :

```
/*
public function afficher() {
    echo "<p>Voiture " . $this->immatriculation . " de la marque " . $this->marque
    . " (couleur " . $this->couleur . ")</p>";
}
*/
```

Il faut aussi corriger la classe appelée dans les “**setFetchMode()**” pour qu’elles créent des objets **ModelVoiture** :

```
public static function getAllVoitures() {
    $sql = "SELECT * FROM voiture";
    $rep = Model::$pdo->query($sql);
    $rep->setFetchMode(PDO::FETCH_CLASS, 'ModelVoiture');
    return $rep->fetchAll();
}

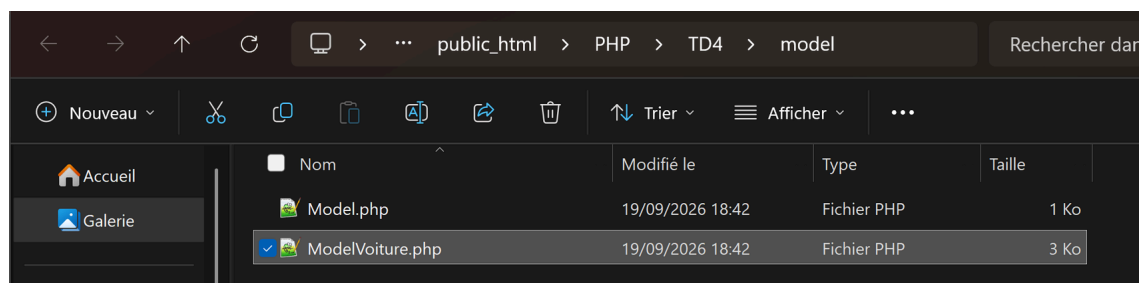
public static function getVoitureByImmat($immat) {
    $sql = "SELECT * from voiture WHERE immatriculation=:nom_tag";
    // Préparation de la requête
    $req_prep = Model::$pdo->prepare($sql);

    $values = array(
        "nom_tag" => $immat,
        //nomdutag => valeur, ...
    );

    // On donne les valeurs et on exécute la requête
    $req_prep->execute($values);

    // On récupère les résultats comme précédemment
    $req_prep->setFetchMode(PDO::FETCH_CLASS, 'ModelVoiture');
    $tab_voit = $req_prep->fetchAll();
    // Attention, si il n'y a pas de résultats, on renvoie false
    if (empty($tab_voit))
        return false;
    return $tab_voit[0];
}
```

On déplace “**ModelVoiture.php**” et “**Model.php**” dans “**model/**”, puis “**Conf.php**” dans “**config/**” :



Comme “**Conf.php**” n’est plus dans le même dossier, on corrige le chemin relatif de l’include dans “**Model.php**” :

```
<?php
require_once '../config/Conf.php';

class Model {
```

Exercice 2

Création de la vue “**view/voiture/list.php**”, elle affiche toutes les voitures contenues dans la variable **\$tab_v** :

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8">
    <title>Liste des voitures</title>
  </head>
  <body>
    <?php
      foreach ($tab_v as $v)
        echo '<p> Voiture d\'immatriculation ' . $v->getImmatriculation() . '</p>';
    ?>
  </body>
</html>
```

Exercice 3

Création du contrôleur “**controller/ControllerVoiture.php**” :

```
<?php
require_once ('../model/ModelVoiture.php'); // chargement du modèle
$tab_v = ModelVoiture::getAllVoitures(); //appel au modèle pour gerer la BD
require ('../view/voiture/list.php'); //redirige vers la vue
?>
```

On teste en appelant l’URL “**.../controller/ControllerVoiture.php**” :

```

Voiture d'immatriculation 00-000-00.
Voiture d'immatriculation AA-000-AA.
Voiture d'immatriculation XX-999-YY.
```

Question : « Avez-vous compris l'ordre dans lequel PHP exécute votre code ? »

Réponse : Oui, **PHP** lit le fichier de haut en bas : il charge d'abord le modèle avec le **require_once**, puis il appelle **getAllVoitures()** qui récupère les voitures dans la **BDD** et les met dans **\$tab_v**, et à la fin il fait le **require** de la vue qui affiche **\$tab_v**.

Question : « Est-ce que ce code vous semble similaire à l'ancien fichier lireVoiture.php ? »

Réponse : Oui, le résultat est le même que "**lireVoiture.php**", sauf que le code est maintenant découpé en 3 : le modèle (accès à la **BDD**), la vue (l'affichage **HTML**) et le contrôleur qui fait le lien entre les deux.

Exercice 4

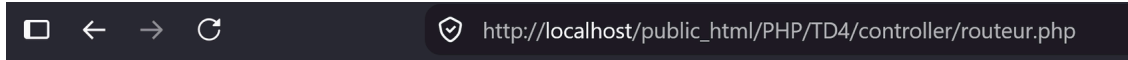
On range le code du contrôleur dans une fonction "**readAll()**" de la classe **ControllerVoiture** :

```
<?php
require_once ('../model/ModelVoiture.php'); // chargement du modèle
class ControllerVoiture {
    public static function readAll() {
        $tab_v = ModelVoiture::getAllVoitures(); //appel au modèle pour gerer la B
        require ('../view/voiture/list.php'); // "redirige" vers la vue
    }
}
?>
```

Création du routeur "**controller/routeur.php**" qui appelle cette méthode :

```
<?php
require_once 'ControllerVoiture.php';
ControllerVoiture::readAll(); // Appel de la méthode statique $action de ControllerVoiture
?>
```

Test avec la page "**.../controller/routeur.php**" :



Voiture d'immatriculation 00-000-00.

Voiture d'immatriculation AA-000-AA.

Voiture d'immatriculation XX-999-YY.

Question : « Avez-vous compris l'ordre dans lequel PHP exécute votre code ? »

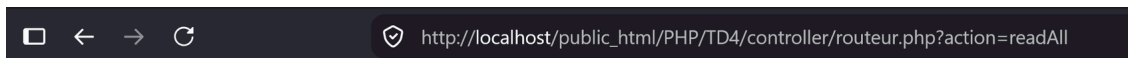
Réponse : Le client demande “**routeur.php**”, qui charge “**ControllerVoiture.php**” (qui charge lui-même le modèle). Ensuite le routeur appelle **ControllerVoiture::readAll()**, qui demande les voitures au modèle puis fait le **require** de la vue.

Exercice 5

On complète la ligne 4 du routeur pour récupérer l'action passée dans l'URL avec **\$_GET** :

```
<?php
require_once 'ControllerVoiture.php';
// On recupère l'action passée dans l'URL
$action = $_GET['action'];
// Appel de la méthode statique $action de ControllerVoiture
ControllerVoiture::$action();
?>
```

On teste en ajoutant “**action=readAll**” dans l'URL au format query string :



Voiture d'immatriculation 00-000-00.

Voiture d'immatriculation AA-000-AA.

Voiture d'immatriculation XX-999-YY.

Question : « Avez-vous compris l'ordre dans lequel PHP exécute votre code ? »

Réponse : Oui, le routeur récupère l'action dans l'URL (**\$_GET['action']** vaut “**readAll**”), puis il appelle la méthode statique qui a le même nom avec

ControllerVoiture::\$action(). Ensuite c'est comme avant : le contrôleur appelle le modèle puis la vue. On obtient la même page que l'ancien **"lireVoiture.php"**.

Exercice 6

Création de la vue **"view/voiture/detail.php"** qui affiche les détails de la voiture **\$v** comme l'ancienne fonction **"afficher()"** :

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8">
    <title>Détail de la voiture</title>
  </head>
  <body>
    <?php
      echo "<p>Voiture " . $v->getImmatriculation() . " de la marque " . $v->getMarque()
        . " (couleur " . $v->getCouleur() . ")</p>";
    >
    <a href="routeur.php?action=readAll">Retour à la liste</a>
  </body>
</html>
```

Ajout de l'action **"read()"** dans le contrôleur, elle récupère l'immatriculation dans l'URL, appelle **"getVoitureByImmat()"** et met la voiture dans **\$v** avant d'appeler la vue :

```
public static function read() {
    $immat = $_GET['immat'];
    $v = ModelVoiture::getVoitureByImmat($immat);
    if ($v === false) {
        require ('../view/voiture/error.php');
    } else {
        require ('../view/voiture/detail.php');
    }
}
```

Test avec **"routeur.php?action=read&immat=AA-000-AA"** :



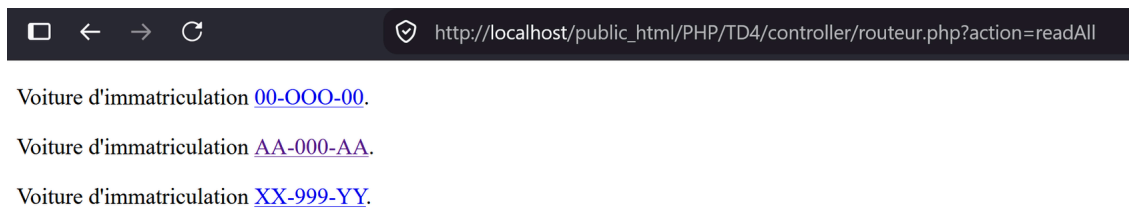
Voiture AA-000-AA de la marque Peugeot (couleur Rouge)

[Retour à la liste](#)

Ajout des liens cliquables sur les immatriculations de “list.php” :

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8">
    <title>Liste des voitures</title>
  </head>
  <body>
    <?php
      foreach ($tab_v as $v)
      <?php
        echo 'Voiture d\'immatriculation <a href="routeur.php?action=read&immat=' . $v->getImmatriculation() . ">";
      <?php
    </body>
  </html>
```

Résultat, les immatriculations sont maintenant des liens vers la vue de détail :



Voiture d'immatriculation [00-000-00](#).

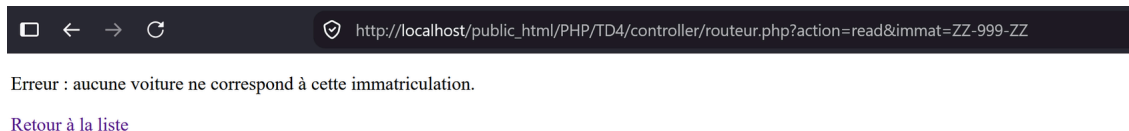
Voiture d'immatriculation [AA-000-AA](#).

Voiture d'immatriculation [XX-999-YY](#).

Pour gérer les immatriculations qui n’existent pas, création de la vue “view/voiture/error.php” :

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8">
    <title>Erreur</title>
  </head>
  <body>
    <p>Erreur : aucune voiture ne correspond à cette immatriculation.</p>
    <a href="routeur.php?action=readAll">Retour à la liste</a>
  </body>
</html>
```

Dans “read()”, si “getVoitureByImmat()” renvoie **false** on appelle cette vue (voir le **if** plus haut). Test avec une immatriculation qui n’existe pas :



Erreur : aucune voiture ne correspond à cette immatriculation.

[Retour à la liste](#)

Exercice 7

Création de la vue “**view/voiture/create.php**” qui reprend le formulaire “**formulaireVoiture.html**” du TD1. Le formulaire est envoyé au routeur, et on rajoute un champ caché pour envoyer “**action=created**” :

```
<form method="get" action="routeur.php">
  <fieldset>
    <legend>Mon formulaire :</legend>
    <input type="hidden" name="action" value="created" />
    <p>
      <label for="marque_id">Marque</label> :
      <input type="text" placeholder="Ex : Opel" name="marque" id="marque_id" required autofocus/>
    </p>
    <p>
      <label for="couleur_id">Couleur</label> :
      <input type="text" placeholder="Ex : Rouge" name="couleur" id="couleur_id" required/>
    </p>
    <p>
      <label for="immat_id">Immatriculation</label> :
      <input type="text" placeholder="Ex : 256AB34" name="immatriculation" id="immat_id" required/>
    </p>
    <p>
      <input type="submit" value="Envoyer" />
    </p>
  </fieldset>
</form>
```

Ajout de l'action “**create()**” qui affiche ce formulaire, et de l'action “**created()**” qui récupère les données dans la query string, crée une **ModelVoiture**, appelle “**save()**” puis “**readAll()**” :

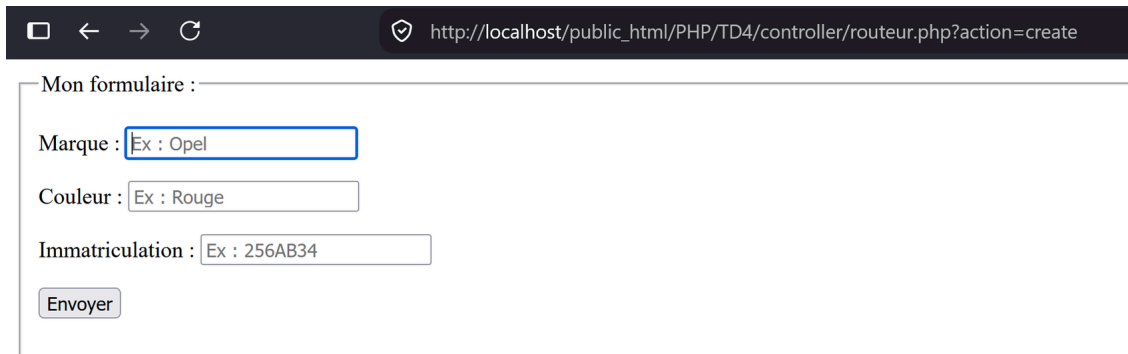
```
public static function create() {
    require ('../view/voiture/create.php');
}

public static function created() {
    $marque = $_GET['marque'];
    $couleur = $_GET['couleur'];
    $immatriculation = $_GET['immatriculation'];

    $v = new ModelVoiture($marque, $couleur, $immatriculation);
    $v->save();

    self::readAll();
}
```

Test de l'action **create** :



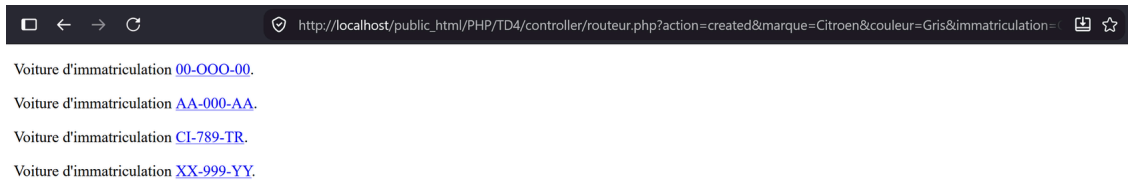
Mon formulaire :

Marque :

Couleur :

Immatriculation :

Test de l'action **created** en donnant directement l'immatriculation, la marque et la couleur dans l'URL :



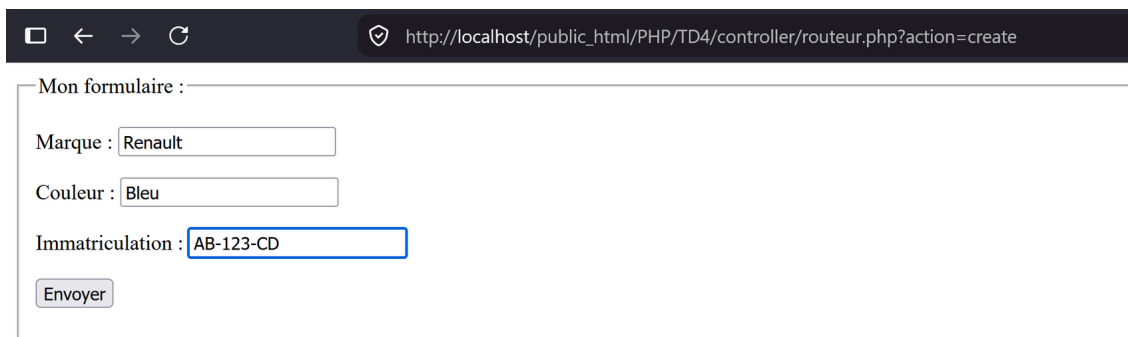
Voiture d'immatriculation [00-000-00.](#)

Voiture d'immatriculation [AA-000-AA.](#)

Voiture d'immatriculation [CI-789-TR.](#)

Voiture d'immatriculation [XX-999-YY.](#)

La voiture “**CI-789-TR**” apparaît bien dans la liste. On teste maintenant le tout avec le formulaire :



Mon formulaire :

Marque :

Couleur :

Immatriculation :

On clique sur “Envoyer”, le formulaire appelle bien l'action **created** (on voit les données dans l'URL) :

Voiture d'immatriculation [00-000-00](#).
Voiture d'immatriculation [AA-000-AA](#).
Voiture d'immatriculation [AB-123-CD](#).
Voiture d'immatriculation [CI-789-TR](#).
Voiture d'immatriculation [XX-999-YY](#).

La voiture a bien été ajouté dans la **BDD** (phpMyAdmin) :

✓ Affichage des lignes 0 - 4 (total de 5, traitement en 0,0003 seconde(s).)

```
SELECT * FROM `voiture`
```

☐ Profilage
[Éditer en ligne] [Éditer] [Expliquer SQL] [Créer le code source PHP] [Actualiser]

☐ Tout afficher | Nombre de lignes : 25 | Filtrer les lignes: Chercher dans ce

Options supplémentaires

				immatriculation	marque	couleur
☐	Éditer	Copier	Supprimer	00-000-00	Mickey	Turquoise
☐	Éditer	Copier	Supprimer	AA-000-AA	Peugeot	Rouge
☐	Éditer	Copier	Supprimer	AB-123-CD	Renault	Bleu
☐	Éditer	Copier	Supprimer	CI-789-TR	Citroen	Gris
☐	Éditer	Copier	Supprimer	XX-999-YY	Peugeot	Rouge

Conclusion

Ce **TD** nous a fait découvrir l'architecture **MVC** : on a réorganisé le code du **TD3** en modèle, vues et contrôleur, avec un routeur qui appelle la bonne action selon l'URL.